📑 جزوه سیستم عامل - فصل ۶: ابزار همگامی (بخش چهارم)

(مانیتورها و عاقبت‌به‌خیری)

۱۰. چرا سمافورها کافی نیستند؟ (مشکلات سمافور)

اگرچه سمافورها (بخش قبل) ابزارهای قدرتمندی هستند، اما استفاده از آن‌ها برای برنامه‌نویسان خطرناک است. یک اشتباه کوچک می‌تواند کل سیستم را فلج کند.

اشتباهات رایج در استفاده از سمافور (توسط برنامه‌نویس):

1. **جابجایی signal و wait:** اگر جای این دو عوض شود، ممکن است چند پردازه هم‌زمان وارد بخش بحرانی شوند (نقض انحصار متقابل).

```
signal(mutex); // اشتباه! اول سیگنال داده
... بخش بحرانی ...
wait(mutex);
```

2. **استفاده از wait تکراری:** اگر به جای signal دوباره wait بزنیم، پردازه تا ابد قفل می‌شود (Deadlock).
3. **فراموش کردن wait یا signal:** انحصار متقابل نقض می‌شود یا سیستم قفل می‌کند.

✓ **راه حل:** استفاده از یک ساختار سطح بالا و امن‌تر به نام **مانیتور (Monitor)**.

۱۱. مانیتورها (Monitors)

مانیتور یک **نوع داده انتزاعی (ADT)** است (مثل یک کلاس در برنامه‌نویسی شی‌گرا) که مدیریت همگامی را "درون خودش" انجام می‌دهد و خیال برنامه‌نویس را راحت می‌کند.

ویژگی‌های کلیدی مانیتور:

1. **کپسوله‌سازی:** متغیرهای مشترک در داخل مانیتور مخفی هستند و فقط توابع خودِ مانیتور می‌توانند به آن‌ها دسترسی داشته باشند.
2. **انحصار متقابل خودکار (نکته امتحانی):** ساختار مانیتور تضمین می‌کند که **در هر لحظه فقط یک پردازه** می‌تواند درون مانیتور فعال باشد. نیازی نیست برنامه‌نویس دستی قفل بگذارد.

ساختار کلی یک مانیتور:



۱۲. متغیرهای شرط (Condition Variables)

مانیتور به تنهایی برای برخی الگوریتم‌های همگامی کافی نیست (چون فقط قفل ساده دارد). برای اینکه پردازه‌ها بتوانند منتظر شرط خاصی بمانند، از **متغیرهای شرط** استفاده می‌کنیم.

- تعریف: ;condition x, y
- روی متغیر شرط فقط دو عملیات مجاز است:

:()x.wait

  - پردازه‌ای که این را صدا بزند، **معلق (Suspend)** می‌شود و در صف انتظار x قرار می‌گیرد.
  - نکته مهم: وقتی پردازه معلق می‌شود، قفل مانیتور را آزاد می‌کند تا بقیه بتوانند وارد شوند.

:()x.signal

- یکی از پردازه‌هایی که روی x منتظر بود را بیدار می‌کند (Resume).

⚠ تفاوت مهم signal در مانیتور و سمافور (سوال کنکور)

- **در سمافور:** اگر signal بزنیم و کسی منتظر نباشد، مقدار سمافور زیاد می‌شود (سیگنال ذخیره می‌شود).
- **در مانیتور:** اگر x.signal() بزنیم و هیچ پردازه‌ای روی x منتظر نباشد، **هیچ اتفاقی نمی‌افتد (سیگنال گم می‌شود).**

استراتژی‌های سیگنال‌دهی (وقتی P به Q سیگنال می‌دهد):

وقتی پردازه P دستور x.signal() را می‌زند و پردازه Q بیدار می‌شود، الان دو نفر فعال داریم. اما مانیتور فقط جای یک نفر است! چه باید کرد؟

1. **سیگنال و انتظار (Signal and Wait):** P منتظر می‌ماند تا Q کارش تمام شود یا دوباره منتظر شود. (هوار - Hoare style)
2. **سیگنال و ادامه (Signal and Continue):** Q منتظر می‌ماند تا P کارش تمام شود و از مانیتور بیرون برود.

۱۳. پیاده‌سازی مانیتور با سمافور (مبحث پیشرفته)

می‌توانیم مکانیسم مانیتور را با استفاده از سمافورها شبیه‌سازی کنیم.

- یک سمافور mutex (با مقدار ۱) برای ورود به مانیتور.
- یک سمافور next (با مقدار ۰) برای اینکه پردازه‌های سیگنال‌دهنده خودشان را روی آن معلق کنند (در روش Signal and Wait).
- یک متغیر next_count برای شمارش تعداد پردازه‌های معلق‌شده.

(جزئیات کد در صفحه ۳۸۲ کتاب آمده است که معمولاً برای حل مسئله داده می‌شود نه حفظ کردن).

۱۴. اولویت‌دهی در انتظار (Conditional Wait)

گاهی می‌خواهیم پردازه‌ها را نه به ترتیب ورود (FIFO)، بلکه بر اساس **اولویت** بیدار کنیم.

- دستور: x.wait(c)

- پارامتر c (عدد صحیح) اولویت است.
- وقتی x.signal() زده شود، پردازه‌ای که **کمترین مقدار** c را دارد بیدار می‌شود.

مثال: تخصیص‌دهنده منبع (Resource Allocator)

فرض کنید یک منبع داریم و می‌خواهیم آن را به کسی بدهیم که "کوتاه‌ترین زمان استفاده" را لازم دارد.

```
} monitor ResourceAllocator
          ;boolean busy
           ;condition x

} void acquire(int time)
          if (busy)
;x.wait(time) // با اولویت "زمان" منتظر بمان
          ;busy = true
                    {

} ()void release
         ;busy = false
;()x.signal // کسی که کمترین time را داشت بیدار می‌شود
                    {
                        {
```

۱۵. عاقبت‌به‌خیری (Liveness)

تا اینجا روی ابزارهایی تمرکز کردیم که "انحصار متقابل" را تضمین کنند (امنیت). اما اگر سیستم امن باشد ولی کار نکند چه؟

- **تعریف Liveness:** مجموعه‌ای از ویژگی‌ها که تضمین می‌کند پردازه‌ها بالاخره پیشرفت می‌کنند و کارشان انجام می‌شود.
- **شکست در Liveness:** یعنی وقتی پردازه در یک وضعیت گیر می‌کند (مثل حلقه بی‌نهایت انتظار).

مثال‌های شکست Liveness:

1. **انتظار نامتناهی (Indefinite Waiting):** پردازه در صف منتظر است و هیچ‌وقت نوبتش نمی‌شود (گرسنگی).
2. **بن‌بست (Deadlock):** دو یا چند پردازه منتظر همدیگر هستند و هیچ‌کدام جلو نمی‌روند. (در فصل بعدی کامل بررسی می‌شود).

(پایان فصل ۶ - بخش مانیتورها و Liveness)